

SkillsHub

An AI-Powered Skills Intelligence Platform | Hackathon Problem Statement

The Story

In a software company, developers are scattered across projects — each carrying a unique mix of platforms, languages, frameworks, and tools. Some are novices, some are seasoned experts, and most sit somewhere in between. Today, this knowledge lives in scattered spreadsheets, outdated profiles, or — more often than we'd like to admit — in someone's head.

When HR needs to staff a new project, they shouldn't have to ping ten managers asking *"who knows React and has worked on payment integrations?"* They should just be able to ask. And the answer shouldn't be a dumb keyword match — it should *understand* the question.

Your mission: build a skills intelligence platform that ingests employee data smartly, understands skills the way humans do, and answers HR's questions in natural language — with real reasoning behind every result.

What You're Building

A skills intelligence platform with **two primary user roles**:

HR Team	Manages the skills database. Uploads or imports employee data. Searches using natural language to find the right people for projects, training, or staffing decisions. Reviews and approves AI-extracted profiles.
Employees	Upload their resume or LinkedIn export — the system extracts their skills automatically. Can review and refine their profile, add proficiency levels, projects, and certifications. HR can also update profiles on their behalf.

The Two Hard Problems

Most of this build is straightforward CRUD. The real challenge — and where the hackathon will be won or lost — sits in two places. Get these right, and the rest is plumbing.

Hard Problem #1 · Smart Profile Ingestion

An employee uploads a resume (PDF) or pastes a LinkedIn profile. Your system should **auto-extract** structured skill data — technologies, proficiency levels, years of experience, and projects worked on. The extracted profile goes into a **review queue** where HR (or the employee) can correct mistakes before it's accepted into the database. Bonus: infer related skills the resume doesn't mention explicitly (e.g., someone with 4 years of Next.js clearly knows React).

Hard Problem #2 · Semantic Natural Language Search

HR types a real-world query — not keywords, full sentences. Your system should return ranked, explained matches.

"Who can lead a React project that also needs WebSocket experience?"

"Find me a backend dev in Pune with at least 3 years of Java and any payment gateway integration."

"Senior frontend folks who haven't been on a new project in the last quarter."

Results must include a **match score** and a **plain-English reason** — e.g., *"Rahul — 94% match. Expert in React (5 yrs), led 2 real-time apps using Socket.IO, currently unallocated."* Dumb keyword matching won't cut it.

Core Features (Must-Have)

- 1 **Authentication & role-based access** — separate experiences for HR and employees.
- 1 **AI-powered profile ingestion** — upload a resume (PDF) and the system extracts skills, proficiency, years of experience, and project history into a structured profile. Use Claude's capabilities — this is one of the two centerpieces.
- 1 **Review & approval workflow** — extracted profiles land in a review queue. HR or the employee verifies and refines before the profile is accepted.
- 1 **Skills taxonomy** — capture skills with category (language, framework, platform, tool, domain), proficiency level (novice / intermediate / expert), and years of experience.
- 1 **Natural language search with ranking & explanations** — HR types a real question, gets back a ranked list of people with match scores and human-readable reasoning. This is the second centerpiece.
- 1 **Employee directory & profile view** — browse the database, view individual profiles, see skills and history at a glance.

Stretch Goals (Bonus Points)

- **Skill inference engine** — if someone has Next.js + TypeScript, infer React + JavaScript automatically (with confidence scores).
- **Team builder mode** — HR describes a project ("4-person team for a 3-month healthcare app, need mobile + backend + DevOps") and the system proposes a team with rationale and alternatives.
- **GitHub integration** — pull a developer's public repos and infer active skills from recent commits.
- **Skill gap analysis** — show which skills the company is light on, relative to current or upcoming project needs.
- **Conversational search** — let HR refine results in a chat-style interface ("only show ones available next month").
- **Bulk import** — upload a CSV or folder of resumes and process them in batch.

Out of Scope (Don't Spend Time Here)

- Performance management, payroll, leave, or any other HR functionality unrelated to skills.
- Integration with corporate SSO / Active Directory / Okta — basic auth is fine.
- Mobile apps — web-first is enough.
- Building your own ML model from scratch — use Claude, embeddings, or any LLM-powered approach.

Tech Stack

Free choice. Pick whatever you and Claude work best with. We'll evaluate the *outcome*, not the stack.

Judging Criteria

Criteria	What we're looking for	Weight
Working demo	End-to-end flow — ingest a resume, search, see results	30%
AI capability	Quality of extraction & semantic search reasoning	25%
Architecture & code quality	Clean structure, sensible decisions, readable code	20%
UX & polish	Does it feel like a real product?	15%
Creativity / stretch goals	Anything extra that wows us	10%

What to Submit

- 1 **GitHub repo** — clean commits and a clear README. Setup instructions matter — we should be able to run it.
- 1 **A short presentation (5–8 slides)** covering: problem understanding, your approach, architecture, demo screenshots, and what you'd build next.
- 1 **A 2–3 minute demo video** (optional but highly recommended) showing the end-to-end flow.

Pro Tips

- **Win on the two hard problems.** Polished extraction + smart search beats a feature-loaded build with weak AI.
- **Let Claude do the heavy lifting** on extraction prompts, semantic ranking, and reasoning — but *you* drive the architecture decisions.
- **Seed data matters.** Bring 10–15 realistic-looking employee profiles so judges can see your search actually work.
- **Show your reasoning.** When search returns results, the *why* is more impressive than the *who*.

Good luck — go build something awesome!